



AquaGrid

An AI that forecasts an island's water demand, schedules desalination against the solar forecast, harvests the rain, and catches failures before they get expensive.

Rishik · Neekin · Srihan · Ednit · Gowtham

DreamHacks 2026 · Track 2 – AI, Automation & Logic

Water and energy are the same problem

Fresh water is rain when it falls, and a desalination plant when it isn't — and that plant is the **largest electrical load on the island**.

Its power is solar by day, diesel by night. The diesel arrives **by barge every few weeks**, whether or not you are running low.

Noon, clear sky

nearly free

02:00

imported diesel

The tank

holds a day or two

So every hour is a decision — and it **can** be deferred, if you know what tomorrow looks like.

Almost nowhere is that decision made

It is delegated to a **wall timer**, set to overnight hours inherited from mainland off-peak electricity tariffs.

On a solar island that is exactly backwards. Those are the hours with no sun.

Tuvalu makes it concrete. The freshwater lens is thin and salt-contaminated, so the island runs on rain.

In **2011 a drought left Funafuti with days of water** and a declared state of emergency. The plant was not too small — **nobody could see far enough ahead to act early.**

One forecast, shared by everything

01 Predict Demand, 48 h ahead	02 Decide 8 rules, 12 h lookahead	03 Explain A reason for every hour	04 Detect Failures nobody would see
--	--	---	--

45.3_{L/h}

mean absolute error

0.987

R^2

89%

better than predicting the average

Chronological split — tested on the 60 days that came **after** the ones it learned from.

78% less diesel — and something better

STRATEGY	DIESEL	COST	CO ₂	FUEL LASTS	OUTCOME
AquaGrid	15.3 L	\$25	41 kg	39 days	holds
Fixed timer 01–05	70.0 L	\$112	188 kg	8.6 days	dry -3.4 d
Reactive < 80%	79.2 L	\$127	212 kg	7.6 days	dry -4.4 d

300 litres of fuel, **12 days to the barge**. Burning too much diesel and running out of diesel are **different failures**.

It all runs on the phone

THE APP	React Native	Expo · one screen, 18 components, charts hand-drawn in SVG
THE BRAIN	RandomForest, bundled in	trained offline in scikit-learn, shipped inside the app — no server call to predict
THE WEATHER	Open-Meteo	the phone fetches its own solar, temperature, rain and wind · no API key
THE VOICE	Groq · GPT-OSS 120B	open-weight LLM writes the briefing and answers questions · optional

Scheduling, rainwater, battery dispatch, every detector and every chart are computed **on the device**. There is no backend — an app that needs a server to run a pump fails exactly when an island is cut off.

We ship the model, not its predictions

Every feature is categorical except temperature — so the trained forest is evaluated over its **entire input grid** and travels inside the app as a table.

15,120

cells — the whole model, not a sample of it

5.5_{L/h}

fidelity loss against its own 45.3

0.011_{ms}

72 predictions, on the phone

So it forecasts for islands the laptop never saw — with no ML runtime, no inference API, and no signal required.

RandomForest • scikit-learn

Trained by us. Makes

every

decision.

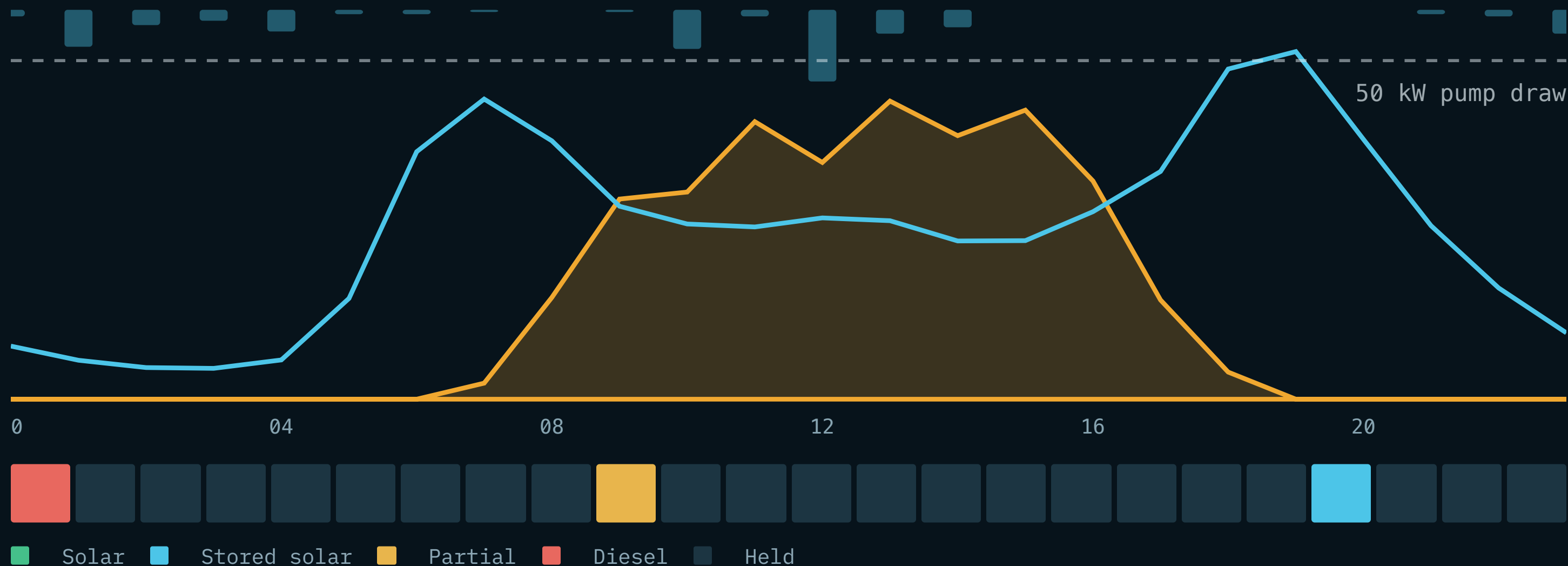
GPT-OSS 120B • Groq

Open-weight, optional. Makes

none

— remove it and no number changes.

One day, as the scheduler sees it



Eight rules, and four ways to fail

0 Storm prep	1 Emergency	2 Rain hold	3 Free solar
4 Stored solar	5 Pre-emptive hybrid	6 Reluctant diesel	7 Hold

Highlighted three act on information the plant **does not have yet**. Rain supplied **24%** of all water; the battery carried **66 kWh** into the evening peak.

Burst pipe

caught in 2 h

Dirty panels

13% below irradiance

Fouling membranes

clean due in 40 days

Fuel

dry before the barge

All four the same idea: what the plant **does** versus what the forecast says it **should**.

Demo

- | | |
|---|--|
| 1 Current decision — read the reason line | 2 Chart & timeline — green free, cyan stored, amber deliberate |
| 3 Savings, then fuel against the barge | 4 Simulate burst pipe — the alert fires |

“Caught from the forecast residual, not a threshold — no human would have noticed for hours.”

Demo

5 **Simulate cyclone** — the objective flips

6 Change island — everything recomputes on-device

7 Ask the operator, then a follow-up

8 **Airplane mode** — nothing stops working

Built for an island with no reliable connectivity. **End there.**

The code

755

lines in the decision layer

0

React imports in it – so it tests without a phone

node
src/logic/test.js

three strategies, the leak, the table fidelity

ml/	Train	synthetic history, forest, lookup-table export
logic/	Decide & detect	scheduler, baselines, leak, assets, fuel, advisory
lib/	Reach out	location, weather, the optional LLM
components/	Present	18 components, charts hand-drawn in SVG

APIs, and everything written down

Open-Meteo Forecast

solar, temp, rain, wind ·
no key

Open-Meteo Geocoding

name → coords + population ·
no key

Groq

open-weight LLM · optional

Nothing else touches the network. No maps SDK, no analytics, no backend of our own.

Website

charts generated from the real
forecast, not screenshots

35_{pages}

team handbook – every file, every
tuned value

Open

github.com/sbongula/aquagrid



Everything is computed, not asserted.

Three schedulers over the same forecast and the same physics. A model tested on the days that came after. A leak detector given a real injected fault. Reykjavík sits in the results at 23% next to Apia at 80% — because a smaller honest number beats a larger one we cannot defend.

github.com/sbongula/aquagrid
sbongula.github.io/aquagrid